ANALYSIS OF ALGORITHMS / PRESENTATION I

Adam
Adaptive Moment Estimation

How does Adam behave across objective landscapes, and what changes when we compare it with Momentum and AMSGrad?

Dario Pomasqui
Yachay Tech University



PROJECT RESOURCES

1. OVERLEAF MANUSCRIPT
2. GITHUB REPOSITORY
3. PRESENTATION VIDEO

Why adaptive optimization?

0:50

Optimization repeatedly adjusts parameters to reduce an objective.

Problem addressed	Adam's idea	Why study it?
A single global step size can be awkward when coordinates have different gradient scales or the objective has uneven geometry.	Smooth recent gradients for a direction; use recent squared gradients to scale each coordinate's step.	Adaptive optimizers are widely used in machine learning. Understanding their updates and limits matters beyond calling a library.

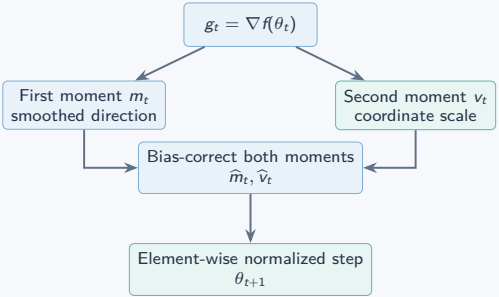
Research question: On deterministic test functions, how do Adam's convergence observations and cost compare with SGD+Momentum and AMSGrad?

Scope: analytic objectives and a prescribed gradient sequence; not a neural-network training benchmark.

One Adam step: direction, scale, correction

$$g_t = \nabla f(\theta_t), \quad m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t \odot g_t$$
$$\widehat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \widehat{v}_t = \frac{v_t}{1-\beta_2^t}$$
$$\theta_{t+1} = \theta_t - \alpha \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}$$

State starts at $m_0 = v_0 = 0$; increment t before correction. Square, square root and division are element-wise. ϵ is **outside** the square root.



Repeat with the new parameters

Adam in pseudocode: the main algorithm

Input: objective f, gradient ∇f, initial vector θ_0, rate α, $\beta_1, \beta_2, \epsilon$, limit T
Initialize: $m_0 \leftarrow \mathbf{0}$, $v_0 \leftarrow \mathbf{0}$
For $t = 1, \ldots, T$:
  $g_t \leftarrow \nabla f(\theta_{t-1})$
  $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$
  $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2)(g_t \odot g_t)$
  $\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$; $\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$
  $\theta_t \leftarrow \theta_{t-1} - \alpha \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$
  Stop if configured criterion is met
Return: θ_t

Algorithmic paradigm

Iterative, first-order, gradient-based optimization. Adam adapts each coordinate using exponential moving averages of gradient moments. It supports stochastic gradients; these experiments use deterministic objectives.

Adam complexity

Update time: $O(d)$ per step, given the gradient.

Optimizer state: m_t, v_t, so $O(d)$ auxiliary memory.

d is the number of parameters. Gradient-computation cost is additional; model and parameter storage are excluded from optimizer auxiliary state.

C++17 implementation of Adam


```cpp
if (t == 0) { m.assign(d, 0.0); v.assign(d, 0.0); }
++t;
const double bc1 = 1.0 - std::pow(beta1, t);
const double bc2 = 1.0 - std::pow(beta2, t);
for (size_t i = 0; i < d; ++i) {
    m[i] = beta1*m[i] + (1-beta1)*grads[i];
    v[i] = beta2*v[i] + (1-beta2)*grads[i]*grads[i];
    const double mhat = m[i] / bc1;
    const double vhat = v[i] / bc2;
    const double update = learning_rate * mhat /
        (std::sqrt(vhat) + epsilon);
    params[i] -= update;
}
```

Project design

`Optimizer::step` accepts parameter and gradient vectors.

Objective functions return $f(\theta)$ and fill a gradient vector.

`reset()` clears state between runs.

Adam settings: $\beta_1 = .9$, $\beta_2 = .999$, $\epsilon = 10^{-8}$; α varies by objective.

Safety checks

Finite positive learning rate; valid β and ϵ; matching dimensions; finite inputs and updates. Invalid cases throw exceptions.

Tests check known gradients, first updates, reset, invalid hyperparameters, dimension changes and non-finite gradients.

Experiments: Adam and comparison methods

1:00

Test	Start $\to$ target	Stop rule
Quad 2D	$(5, -3) \to (0, 0)$	$\|g\|_2 < 10^{-8}$
Quad 10D	10D vector $\to \mathbf{0}$	$\|g\|_2 < 10^{-8}$
Rosenbrock	$(-1.5, 2) \to (1, 1)$	$\|g\|_2 < 10^{-6}$
Rastrigin 2D	$(2.5, 2.5) \to (0, 0)$	$\|g\|_2 < 10^{-6}$
Reddi	$x_0 = 0 \to$ target -1	300k fixed steps

Study design

Main algorithm: **Adam**.
Comparisons: SGD+Momentum, $u_t = \beta u_{t-1} + g_t$; AMSGrad, $v_t^{\max} = \max(v_{t-1}^{\max}, v_t)$.

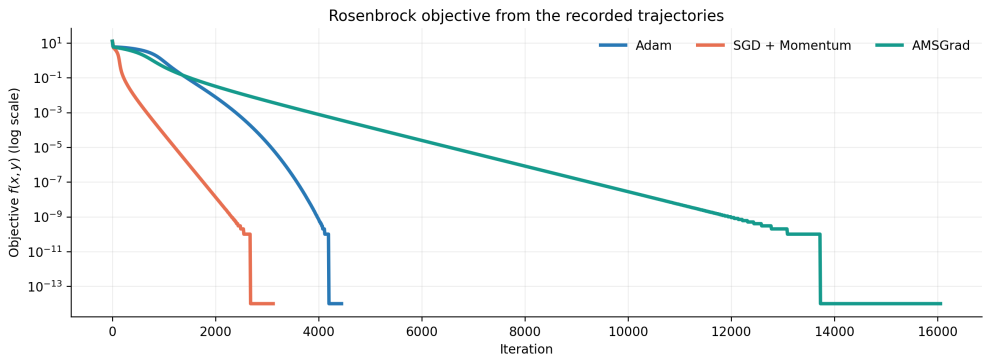
Fairness choices

Same start, cap and stopping rule. Rates (Adam/Momentum/AMSGrad): quadratic .01/.01/.01; Rosenbrock .01/.001/.01; Rastrigin .05/.005/.05. No equal tuning budget.

C++17 / CMake runner • CSV: loss, gradient norm, distance, coordinates • plots from CSV • CTest: 1/1 passed

Rosenbrock: Adam and comparison results

1:20



4,443
Adam updates to $\|\nabla f\| < 10^{-6}$

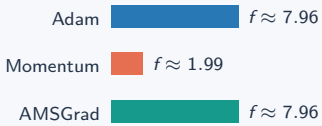
3,118
SGD + Momentum updates

16,057
AMSGrad updates

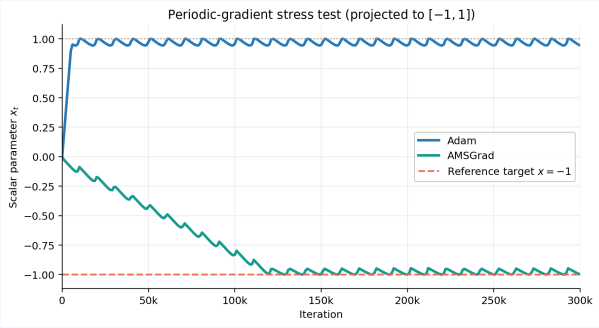
Adam on multimodal and prescribed gradients

Rastrigin, start $(2.5, 2.5)$

All runs satisfy $\|\nabla f\| < 10^{-6}$, but the known global objective is 0.



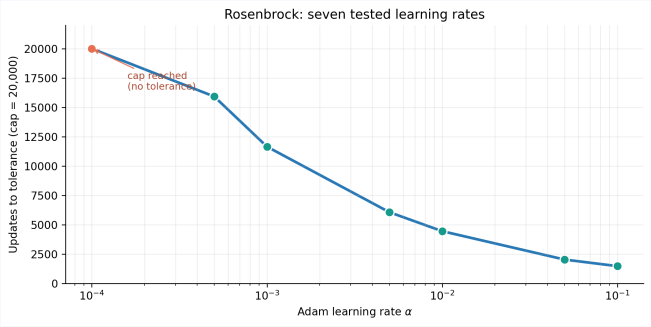
One start illustrates local minima; it does not estimate global success rates.



Finite projected schedule from $x_0 = 0$; $\alpha = .01$, $\beta_1 = .9$, $\beta_2 = .99$. Endpoints: Adam 0.9454, AMSGrad -0.9982 (target -1).

Adam learning-rate sensitivity

1:00



Adam on Rosenbrock; one fixed start. The lowest tested rate reaches the cap without meeting tolerance.

Interpretation

The learning rate changes Adam's observed progress on this objective. The smallest tested rate reached the iteration cap. This supports sensitivity in the tested grid only.

Timing is from one run, not a performance benchmark.

Adam: conclusions and limits

Main contribution	Evidence	Comparison role
Adam is implemented in C++17 for parameter vectors, with bias correction and numerical checks.	Outcomes change with the objective and rate; a small gradient did not imply the Rastrigin global minimum.	Momentum and AMSGrad are comparison baselines. No method is shown to be universally superior.

Limits: small deterministic tests; one start per objective; no equal tuning budget; single-run timing; finite Reddi construction. No universal superiority or convergence claim.

References: D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," ICLR (2015). S. J. Reddi, S. Kale, and S. Kumar, "On the Convergence of Adam and Beyond," ICLR (2018). B. T. Polyak, "Some Methods of Speeding up the Convergence of Iteration Methods" (1964).

Thank you

Questions?